

TERRAFORM STUDY SERIES · VOLUME TWO

Practice Lab



40 drills · full answers · solo-proof

Broken code, plan predictions, output reading, scenarios, refactors, traces — every exercise solvable from **Book 1 (Terraform Master)**, every solution fully worked. Attempt before hint, hint before solution.

A Technical Book · INK & AMBER Series

Companion to **Book 1 — Terraform Master** (same 24 topics, same order)

● WARM-UP ×15 ● DRILL ×15 ● BOSS ×10



Training Map

Three bands, Book 1 order inside each. Every row jumps to the exercise; every exercise links to its solution — and solutions live pages away, so no spoilers.

I FRONT MATTER

01	Cover	start
02	Training map (this page)	you are here
03	Training route	read first
04	How to train	rules

II WARM-UPS (Book 1 Ch 1–15)

05	ex-01 re-apply · ex-02 version	laC · facts
06	ex-03 pillars · ex-04 apply order	why · core
07	ex-05 leaked keys · ex-06 block labels	provider · VPC
08	ex-07 init · ex-08 plan diff	init · plan
09	ex-09 pins · ex-10 first command	providers · CLI
10	ex-11 lost state · ex-12 isolate prod	state · backend
11	ex-13 lock · ex-14 rename · ex-15 fmt	lock · mv · fmt

III DRILLS (Book 1 Ch 16–23)

12	ex-16 graph · ex-17 hardcode→vars	graph · vars
13	ex-18 precedence · ex-19 three envs	vars · workspaces
14	ex-20 drop middle · ex-21 →for_each	count · keys
15	ex-22 secret output · ex-23 prov myth	outputs · prov
16	ex-24 reuse · ex-25 console drift	modules · drift
17	ex-26 refresh · ex-27 adopt VPC	refresh · import
18	ex-28 missing alias · ex-29 TF VAR types	regions · env vars
19	ex-30 safe access	one()

IV BOSSES + ANSWERS

20	ex-31 race · ex-32 wrong value	lock · debug
21	ex-33 scale rename · ex-34 half-built	keys · prov fail
22	ex-35 extract module · ex-36 destroy	modules · teardown
23	ex-37 cross-VPC SG · ex-38 remote state	peering · stacks
24	ex-39 provider bump · ex-40 FINAL BOSS	upgrade · chain
25	Solutions warm-ups	sol-01–15
26	Solutions drills	sol-16–30
27	Solutions bosses	sol-31–40
28	Scorecard	❏ end

Warm-ups → Drills → Bosses → Capstone

Book 1 taught you the tools; this book puts reps in. **Warm-ups** (ex-01-15) check one fact each from **Book 1, Chapters 1-15** — if any feel hard, re-read that chapter first. **Drills** (ex-16-30) combine two ideas from **Chapters 16-23** (precedence, keys, sharing, adopting). **Bosses** (ex-31-40) are incidents: races, half-built boxes, refactors at scale. The **FINAL BOSS** rebuilds the **Chapter 24** chain cold.

 **THE RULES (FROM HOW TO TRAIN)**

Attempt → **hint** → **solution**. Predict every plan BEFORE running it. No console until the diff is written down. Every exercise links its solution (→ **Solution** style) — solutions live pages away, so wandering eyes learn nothing.

▣The 7 drill types (each appears ≥3×)

- T1 Find the bug · T2 Predict the plan**
Broken HCL or a config+state pair — write the fix or the exact +/-/~ diff.
- T3 Read the output · T4 Scenario**
Real terminal text to diagnose; client briefs to design + code.
- T5 Command choice · T6 Refactor**
Four tempting options, one right; reshape code, result unchanged.
- T7 Trace the flow**
"What exactly happens, in order, when...?" — narrate the machinery.

▣Where the bands live

Band	Exercises	Book 1 ground	Done when
 Warm-ups	ex-01-15	Ch 1-15, one fact each	13/15 without hints
 Drills	ex-16-30	Ch 16-23, two ideas each	solutions match on WHY, not just answer
 Bosses	ex-31-40	incidents + final chain	scorecard all green

Rules, Bands & the 7 Drill Types

Training beats re-reading. Each exercise is solvable from **Book 1** alone — chapter named on every header — and each solution teaches the reasoning, not just the answer.

The three rules

- **Attempt before hint.** Write something — even a wrong diff teaches.
- **Hint before solution.** One nudge is usually enough.
- **No console until predicted.** Plan on paper, then run.

The bands

-  **Warm-up** — one fact, 2 minutes.
-  **Drill** — two ideas combined, 5–8 minutes.
-  **Boss** — incident or refactor, 10–15 minutes.

▣ Anatomy of an exercise page

```
EXERCISE NN · <topic> [  WARM-UP · Book 1 Ch N ]  
└ THE TASK — scenario + exact deliverable ───────────┐  
└ HINT — one nudge, never the answer ───────────┐  
└ → Solution on page NN ───────────┐
```

▣ Anatomy of a solution

```
 THE ANSWER — code / diff / diagnosis (fully worked)  
 WHY — 2–4 sentences of reasoning (the real lesson)  
 TRAP — the tempting wrong answer, and why it bites  
 EXAM — one testable line
```

SCORING YOURSELF

Mark the **scorecard** honestly: **green** = solved cold, **amber** = needed the hint, **red** = read the solution. Reds go back into next week's rotation.

ex-01 • ex-02 warm-ups

ex-01 THE TASK (T2 • Predict the plan)

You applied this config yesterday afternoon and went home. Nothing changed overnight — no edits, no console clicks, no teammate applies. **This morning you run terraform apply again.** Write down the exact plan tail: how many to add / change / destroy — and the one sentence Terraform prints when reality already matches code.

```
resource "aws_vpc" "main" { cidr_block = "10.0.0.0/16" }
# state: { "type": "aws_vpc", "name": "main", "id": "vpc-0ab123456" }
```

HINT

Same code → same infra — that's the whole promise of **Book 1, Chapter 1**. What does "no diff" look like in plan language? (Two words, then three zeros.)

ex-02 THE TASK (T3 • Read the output)

A teammate pastes this into the PR and asks "are we on the right toolchain?" **Answer three things:** (1) the Terraform CLI version, (2) the AWS provider version — and whether it's locked or floating, (3) the single command that produced the lock, and the filename it wrote.

```
$ terraform version
Terraform v1.9.8
on linux_amd64
+ provider registry.terraform.io/hashicorp/aws v5.31.0

$ ls -la
.terraform.lock.hcl
main.tf  versions.tf
```

HINT

Two versions hide in the first output; the second output names the guarantee. Which file makes your teammate's init download byte-identical plugins? (**Book 1, Chapter 7**.)

→ **Solutions sol-01 • sol-02**

Follow-up rep — drift breaks the streak

Same VPC, but last night someone tagged it by hand in the console (Owner=ops). **Predict:** does the morning plan still print 0/0/0? Write the new tail — and the one command that re-blesses reality without changing code.

```
# code: no tags block • real VPC: tags = { Owner = "ops" }
$ terraform plan # → ? to add, ? to change, ? to destroy
```

EXAM LENS

Associate loves idempotence phrasing: "second apply, no changes" → **No changes. Infrastructure matches.** Toolchain questions test lock-file purpose, never version trivia.

ex-03 • ex-04 warm-ups

ex-03 THE TASK (T5 • Command choice)

Your team runs AWS + Cloudflare + GitHub. A manager says "CloudFormation is free and deep — why pay the Terraform learning cost?" **Pick the pillar that answers him, then kill the other three options with one line each:**

A) Coding best practices B) Platform-agnostic C) Massive community D) Speed & reliability

```
# your stack in one apply:
resource "aws_s3_bucket" "a" { bucket = "x" }
resource "github_repository" "c" { name = "infra" }
```

HINT

Which pillar dies the moment you add a second API? (**Book 1, Chapter 2**, pillar 2 — and its "we're AWS-only" objection.)

ex-04 THE TASK (T7 • Trace the flow)

You run apply on a config with one VPC and one subnet referencing it. **Number these events 1–6 in true execution order:** ☐ provider authenticates to AWS ☐ Core builds the dependency graph ☐ subnet created ☐ plan diff computed ☐ VPC created ☐ HCL parsed.

```
resource "aws_vpc" "main" { cidr_block = "10.0.0.0/16" }
resource "aws_subnet" "a" {
  vpc_id = aws_vpc.main.id # ← the edge the graph follows
  cidr_block = "10.0.1.0/24"
}
```

HINT

Reading always precedes planning; planning precedes touching anything. Who creates — Core or the plugin? (**Book 1, Chapter 4**.)

→ [Solutions sol-03 • sol-04](#)

Follow-up rep — who runs what

Your apply fails with Error: configuring Terraform AWS Provider: no valid credential sources. **Whose job failed — Core's or the provider plugin's?** Name the failing phase (parse / graph / plan / auth / create) and the cheapest fix that keeps secrets out of git.

```
$ terraform apply
Error: configuring Terraform AWS Provider: no valid credential sources
# provider block carries region only. Shell carries nothing yet.
```

EXAM LENS

Platform-agnostic is the #1 "why Terraform" answer. Core-vs-plugin: any question with "who authenticates / who builds the graph" splits exactly on that line.

ex-05 • ex-06 warm-ups

ex-05 THE TASK (T1 • Find the bug)

This `provider.tf` was just committed to a public repo. **Two things are wrong** — one is a secret leak, one will break every non-default-AWS setup. Name both and write the fixed block.

```
# provider.tf (COMMITTED — do not do this)
provider "aws" {
  region    = "us-east-1"
  access_key = "AKIAIOSFODNN7EXAMPLE"
  secret_key = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
}
```

HINT

Where should secrets live instead — and what decides the endpoint/region when the block is minimal? (**Book 1, Chapter 5** — three credential methods.)

ex-06 THE TASK (T1 • Find the bug)

`terraform validate` rejects this block. **Three label errors hide in four lines** — the keyword, the type, and the local name are each wrong in a different way. Rewrite it so `aws_vpc.main.id` resolves.

```
resources "aws-vpc" "Main" {
  cidr_block = "10.0.0.0/16"
}
```

HINT

Anatomy is resource "TYPE" "name" — check plurals, dashes vs underscores, and case. (**Book 1, Chapter 6**.)

→ [Solutions sol-05 • sol-06](#)

Follow-up rep — read the address

Given the fixed block from ex-06, **write the reference expressions** a subnet and an output would use: (1) the VPC's ID inside `aws_subnet.a`, (2) the CIDR inside an output named `vpc_cidr`. Then state the rule that makes both resolve.

```
resource "aws_subnet" "a" { vpc_id = ???   cidr_block = "10.0.1.0/24" }
output "vpc_cidr" { value = ??? }
```

EXAM LENS

Address syntax `TYPE.NAME.attr` is tested directly. Secrets-in-provider is the classic "what's wrong" exhibit — env vars or SSO, always.

ex-07 • ex-08 warm-ups

ex-07 THE TASK (T3 • Read the output)

Fresh clone, first init. **Three questions:** which two providers installed (and versions)? What file now guarantees your teammate gets byte-identical plugins? What directory holds the downloaded binaries — and does it belong in git?

```
$ terraform init
Initializing provider plugins...
- Installing hashicorp/aws v5.31.0...
- Installed hashicorp/aws v5.31.0 (signed by HashiCorp)
- Installing hashicorp/local v2.5.1...
Terraform has been successfully initialized!
```

HINT

Binaries, lock file, ignore canon — three artifacts, three fates. (**Book 1, Chapter 7.**)

ex-08 THE TASK (T2 • Predict the plan)

State holds `web[0]` and `web[1]`. You change `count = 3` → `count = 2` and shrink the instance type. **Write the full plan tail** — and mark which line destroys.

```
resource "aws_instance" "web" {
  count = 2 # was 3
  instance_type = "t3.small" # was t3.micro
}
```

HINT

Two change kinds here: a type swap (which symbol?) and a count shrink (which index dies?). Colors: **+** add, **-** destroy, **~** change.

→ [Solutions sol-07 • sol-08](#)

Follow-up rep — plan archaeology

A saved `tfplan` file sits in CI artifacts from last night. **Without applying anything**, show what the VPC's planned CIDR was and whether anything was slated for destruction — exact commands, jq included.

```
$ terraform show -json tfplan | jq '.planned_values.root_module.resources[] | {addr: .address, cidr: .values.cidr_block}'
$ terraform show -json tfplan | jq '.resource_changes[] | select(.change.actions[] == "delete") | .address'
```

EXAM LENS

Plan symbols (`+/~/-`) and `plan -out` appear constantly. `init` downloads; `plan` previews; `apply` executes — never in any other order.

ex-09 • ex-10 warm-ups

ex-09 THE TASK (T5 • Command choice)

You pin the AWS provider. **One of these four is the house default — name it, and kill the other three with one line each:**

A) version = ">= 5.0" B) version = "~> 5.0" C) version = "= 5.31.0" D) no version at all

```
terraform {
  required_providers {
    aws = { source = "hashicorp/aws" version = "???" }
  }
}
```

HINT

You want bug-fix updates automatically and major-break surprises never. Which operator means "5.x, never 6.0"? (**Book 1, Chapter 17.**)

ex-10 THE TASK (T5 • Command choice)

brand-new folder, one main.tf, empty backend. **Order these four — exactly one sequence is safe:** ☐ terraform apply ☐ terraform plan -out=tfplan ☐ terraform init ☐ terraform apply tfplan. Then: which step is read-only, and which file makes "reviewed == executed" true?

```
# main.tf exists. .terraform/ does not. No state file. Go.
```

HINT

Binaries before previews, previews before changes; the saved plan is the contract. (**Book 1, Chapters 7-8.**)

→ [Solutions sol-09 • sol-10](#)

Follow-up rep — the 6.0 morning

Overnight the registry published provider 6.0.0 (breaking: tags argument renamed). Teammate A pinned "~> 5.0", teammate B pinned ">= 5.0". Both run fresh init. **Who builds clean, who breaks — and which line in whose file decides it?**

```
# A: version = "~> 5.0" → installs ???
# B: version = ">= 5.0" → installs ??? → plan shows ???
```

EXAM LENS

~> is the only pin operator the exam loves. plan -out=tfplan + apply tfplan is "reviewed == executed" — quote that phrase back.

ex-11 • ex-12 warm-ups

ex-11 THE TASK (T7 • Trace the flow)

Someone `rm`s `terraform.tfstate` on a project managing one VPC — then runs `apply`. **Narrate exactly what Terraform believes at each step** (before `rm` / after `rm` / during `plan` / after `apply`) and name the incident this causes in AWS.

```
# before: config 1 VPC • state 1 VPC (vpc-0ab1) • AWS 1 VPC
$ rm terraform.tfstate
$ terraform apply -auto-approve
```

HINT

State is the ledger, not a suggestion. No ledger → what does "desired vs known" compute? (Book 1, Chapters 9 + 11 — risk ①.)

ex-12 THE TASK (T4 • Scenario)

Client: "dev and prod share one local state file. Last week a dev `apply` nearly touched prod." **Design the fix:** backend type, the two state keys, and the one-line rule that prevents recurrence. No code beyond the backend block.

```
terraform {
  backend "s3" {
    bucket = "???" key = "???"
    region = "us-east-1" dynamodb_table = "???" encrypt = true
  }
}
```

HINT

Separate keys stop the bleeding; separate buckets + locking stop the incident class. (Book 1, Chapter 10 — patterns 1-2.)

→ [Solutions sol-11 • sol-12](#)

Follow-up rep — refresh first

Before narrating ex-11's disaster, a senior says "show me the refresh." **Write the two commands** that (1) preview what reality changed WITHOUT touching state, and (2) print the state's recorded VPC ID for comparison — then state which output proves the ledger is stale.

```
$ terraform plan -refresh-only -out=drift.plan # → shows ???
$ terraform state show aws_vpc.main | grep id # → ???
```

EXAM LENS

"tfstate deleted" → duplicates. "Two envs, one state" → separate keys minimum, separate buckets for prod. Both are favorite scenario stems.

ex-13 · ex-14 · ex-15 warm-ups

ex-13 (T3) — the lock error

Second engineer sees this mid-apply. **What happened, what must she check before touching anything, and the exact release command?**

```
Error: Error acquiring the state lock
Lock Info: ID a1b2-c3 · Operation: apply · Who: ci-runner-4
```

HINT

Read WHO/WHEN/WHAT first; release only the dead. (Book 1, Chapter 11.)

ex-14 (T6) — rename without re-creating

Rename `aws_instance.web` → `aws_instance.app` with **zero downtime and zero rebuild**. Give the two commands in order (ledger first, code second).

```
# state: aws_instance.web (i-012345, live traffic). Goal: same box, new name.
```

HINT

Change the ledger, not the cloud — then let the code match. (Book 1, Chapter 12 — mv.)

ex-15 (T1) — the unformatted block

`fmt -check` fails CI on this file. **Rewrite it canonically** (indent, spacing, alignment) and name the gate flag.

```
resource "aws_vpc" "main"{
cidr_block="10.0.0.0/16"
tags    = {Name="x"}
}
```

HINT

Two spaces, aligned =, sorted keys. (Book 1, Chapter 13.)

→ [Solutions sol-13 · 14 · 15](#)

Follow-up rep — the ledger audit

Before leaving for the day you snapshot state for the runbook. **Give the exact command** that dumps raw JSON to a file, then the `jq` that lists every managed address from that file — no Terraform binary needed for the second step.

```
$ terraform state pull > state-$(date +%F).json
$ jq -r '.resources[] | "\(.type).\(.name)"' state-2026-09-08.json
```

EXAM LENS

`mv` renames tracking, `rm` abandons, `fmt -check` gates CI, `force-unlock` needs a `LOCK_ID` and a corpse. Four commands, four sentences.

ex-16 · ex-17 drills

ex-16 THE TASK (T3 · Read the graph)

apply creates the subnet BEFORE the VPC and fails. The graph SVG shows **no edge** between them. **Diagnose from this block:** why is there no edge, what's the one-line fix, and the command that redraws the proof?

```
resource "aws_vpc" "main" { cidr_block = "10.0.0.0/16" }
resource "aws_subnet" "a" {
  vpc_id = "vpc-0ab123456" # ← pasted from console, not a reference
  cidr_block = "10.0.1.0/24"
}
```

HINT

Edges come from resource.A.x references — a pasted string is invisible to the graph. (Book 1, Chapters 4 + 6.)

ex-17 THE TASK (T6 · hardcoded → variables)

This block works but hardcodes everything. **Refactor to variables.tf + references** with types, one default, one required var with validation — result identical, plan empty.

```
resource "aws_instance" "web" {
  ami = "ami-0c55b159cbfafa1f0" instance_type = "t3.micro"
  tags = { Name = "web-0" Env = "dev" }
}
```

HINT

Four values → four variables; env gets the validation block, the AMI gets no default. (Book 1, Chapter 14.)

→ [Solutions sol-16 · sol-17](#)

Follow-up rep — validate before graph

Before redrawing ex-16's graph, CI wants a zero-cloud syntax gate. **Name the command, show its success output, and state exactly what it does NOT check** (the gap plan still owns).

```
$ terraform validate
Success! The configuration is valid.
# checks: syntax + types + references exist. NOT checked: ???
```

EXAM LENS

Implicit ordering (references) beats depends_on (explicit, last resort). validate is syntax-only — cloud truth needs plan.

ex-18 • ex-19 drills

ex-18 THE TASK (T2 • Predict the winner)

Four layers set env differently. **Predict the applied value, then name the single experiment that proves the full ladder:**

```
variables.tf:      variable "env" { default = "dev" }
terraform.tfvars:  env = "staging"
TF_VAR_env=prod    (exported)
CLI: terraform apply -var="env=qa" -auto-approve
```

HINT

Order: -var > tfvars > env > default. The proof sets all four differently and reads the winner. (Book 1, Chapter 14.)

ex-19 THE TASK (T4 • Scenario)

Client: "one codebase, three envs (dev 1×micro, staging 2×small, prod 3×small), isolated state, no copy-paste." **Write:** the sizing map keyed by workspace, the instance block using it, and the three workspace commands. Then state the one visibility rule that keeps humans safe.

```
# sizing = { dev = {...} staging = {...} prod = {...} }
# current = local.sizing[???] ← the workspace key
resource "aws_instance" "cluster" { count = ??? instance_type = ??? }
```

HINT

terraform.workspace is the key; the safety rule is typed before every apply. (Book 1 workspaces + Vol 2.)

→ [Solutions sol-18 • sol-19](#)

Follow-up rep — console proof

Settle ex-18's ladder dispute without applying anything: **write the one console pipeline** that evaluates the winning value with all four layers loaded, and predict its exact output for the ex-18 setup.

```
$ echo 'var.env' | terraform console -var="env=qa"
"qa" # ← CLI layer wins even in the REPL. Try dropping -var: ???
```

EXAM LENS

Precedence ladder questions give 3+ layers and ask the winner — always trace top-down from -var. Workspaces isolate STATE; variables never do.

ex-20 • ex-21 drills

ex-20 THE TASK (T2 • Predict the drop)

count = 3 → count = 2, and downstream aws_lb_target_group_attachment reads web[2].id. **Write the plan tail AND the downstream error that follows** — then state the one-word reason for_each wouldn't do this.

```
# state: web[0] web[1] web[2] → config: count = 2
# downstream: target_id = aws_instance.web[2].id
```

HINT

Count kills the LAST index; the attachment still points at it. Keys vs indexes — that's the whole chapter. (Book 1, Chapter 20.)

ex-21 THE TASK (T6 • count → for_each)

Three subnets by count, one per AZ, referenced as public[0..2]. **Convert to a for_each map keyed by AZ** — same three subnets, same CIDRs, plan must show the catch (there is one), and you must resolve it without downtime.

```
resource "aws_subnet" "public" {
  count = 3
  cidr_block = cidrsubnet("10.0.0.0/16", 8, count.index)
  availability_zone = data.aws_availability_zones.azs.names[count.index]
}
```

HINT

Keys change identity — [0] vs ["us-east-1a"] — so plan wants rebuilds. Which ledger command preserves them? (Book 1, Chapters 12 + 20.)

→ [Solutions sol-20 • sol-21](#)

Follow-up rep — values() rescue

After ex-21's migration the ALB attachment still reads public[0].id and fails. **Rewrite that single reference** for a for_each map, then give the expression that returns ALL subnet IDs as a list for the output block.

```
# broken: subnet_id = aws_subnet.public[0].id
# fixed single: subnet_id = ???
# fixed all:   output "ids" { value = ??? }
```

EXAM LENS

count vs for_each is the single most-tested Day-2 topic. Index shifts vs stable keys — say those five words and the follow-up writes itself.

ex-22 • ex-23 drills

ex-22 THE TASK (T3 • Read the secret)

CI prints this after output `-raw db_password`. **Two violations in one screenshot** — name both, then give the two correct moves (marking + transport).

```
$ terraform output -raw db_password
Warning: "db_password" is sensitive
p@ssw0rd!      # ← visible in CI logs, copied to Slack
# output block has NO sensitive flag; value piped to chat, not a store
```

HINT

One flag hides CLI display; one habit keeps secrets out of logs. Neither encrypts state — what does? (**Book 1, Chapter 21.**)

ex-23 THE TASK (T1 • The provisioner myth)

A teammate edits the `remote-exec` inline script, re-applies, sees no diff, and declares "Terraform is broken." **Explain why nothing ran** (lifecycle rule), then give the two correct fixes: the immediate recovery AND the architectural cure.

```
resource "aws_instance" "web" {
  # ... unchanged ...
  provisioner "remote-exec" {
    inline = ["sudo apt-get install -y nginx", "sudo systemctl start nginx"]
    # edited just now – apply shows: 0 to add, 0 to change, 0 to destroy
  }
}
```

HINT

Creation-only is the whole doctrine. Recovery is a ledger command; the cure removes the provisioner entirely. (**Book 1, Chapter 22.**)

→ [Solutions sol-22 • sol-23](#)

Follow-up rep — precondition guard

Ex-22's team wants the pipeline to FAIL if anyone removes the sensitive flag. **Write the output block** with a precondition that enforces non-empty + sensitive handling — then state where the check runs (plan or apply) and what the failure looks like.

```
output "db_password" {
  value = aws_db_instance.main.password
  sensitive = true
  precondition { condition = ???   error_message = "???" }
}
```

EXAM LENS

sensitive masks display; state stays plaintext. Provisioners fire once at creation — any "re-run" question answers itself with that rule.

ex-24 • ex-25 drills

ex-24 THE TASK (T4 • Scenario)

Three teams copy-paste the same 60-line VPC block; VPC #4 ships with a different CIDR scheme and breaks peering. **Design the module fix:** module dir layout (3 files), the two inputs it must take, and one root call block. No full code — interface only.

```
# modules/vpc/{ ____, ____, ____ }
# inputs: ____ (string), ____ (map or list)
module "prod_vpc" {
  source = "____"
  ____ = "10.20.0.0/16"
}
```

HINT

main/variables/outputs; cidr + tags-or-AZs. Pin with version for registry, commit hash for git. (Book 1, Chapter 23.)

ex-25 THE TASK (T7 • Trace the drift)

Someone opens the console and opens port 22 to 0.0.0.0/0 on the web SG . Nobody touches code. **Narrate the next plan line by line** — what symbol, on which block — then give the two ways to resolve it (keep vs revert) with exact commands.

```
# code: ingress 443 only • real SG: 443 + 22-open-by-hand
$ terraform plan
```

HINT

Refresh reads REAL first — the diff writes itself. Resolve by code (apply) or by console (revert). (Book 1, Chapter 9.)

→ [Solutions sol-24 • sol-25](#)

Follow-up rep — module outputs

Ex-24's module is accepted. Downstream needs the VPC ID and must NOT reach into module internals. **Write the module's outputs.tf (two values) and the root reference** that wires them into a subnet — wrong answers reach past the interface.

```
# modules/vpc/outputs.tf
output "vpc_id" { value = ??? } # NOT aws_vpc.this.id exposed how?
# root: subnet vpc_id = module.prod_vpc.???
```

EXAM LENS

Root vs child module outputs; ~ means drifted-in-place. `terraform show -json + jq` is the CI interrogation pattern.

ex-26 • ex-27 drills

ex-26 THE TASK (T5 • Command choice)

"Show me drift without changing anything" — and separately, "my tutorial says run terraform refresh." **Pick the right command for each job and retire the legacy one:**

A) terraform refresh B) terraform plan -refresh-only -out=drift.plan C) terraform apply -refresh-only

```
# job 1: preview drift, change nothing
# job 2: update state to match reality (rare, deliberate)
```

HINT

Standalone refresh is legacy; the flag form previews or writes explicitly. (Book 1, Chapters 9 + 16 +  modern note.)

ex-27 THE TASK (T4 • Adopt the VPC)

A hand-built VPC (vpc-0ab123456, 10.0.0.0/16) must come under management with zero downtime. **Write the three steps in order** — config first, then the bind command with exact address + ID, then the command that proves the binding is clean.

```
# step 1: resource "aws_vpc" "main" { cidr_block = "10.0.0.0/16" }
# step 2: terraform _____
# step 3: _____ (must show: 0 to add, 0 to change, 0 to destroy)
```

HINT

Write → bind → prove. The bind takes ADDR then ID; the proof is a quiet plan. (Book 1, Chapter 12.)

→ [Solutions sol-26 • sol-27](#)

Follow-up rep — show the drift

Ex-26's team adopts a rule: every Friday someone posts the drift report. **Write the two-command Friday ritual** (preview file + human-readable summary via show) and state where the artifact is stored so Monday's apply reviews exactly it.

```
$ terraform plan -refresh-only -out=friday.plan
$ terraform show -json friday.plan | jq '.resource_changes[] | {addr: .address, acts: .change.actions}'
```

EXAM LENS

import binds identity; config must already describe the object. -refresh-only previews drift; bare refresh is legacy vocabulary.

ex-28 • ex-29 drills

ex-28 THE TASK (T1 • Find the bug)

Both VPCs landed in us-east-1 and peering failed on overlap — but the code "clearly" creates a DR VPC. **One line is missing.** Name it, place it, and state what plan shows after the fix.

```
provider "aws" { region = "us-east-1" }
provider "aws" { alias = "eu"   region = "eu-west-1" }
resource "aws_vpc" "primary" { cidr_block = "10.0.0.0/16" }
resource "aws_vpc" "dr" { cidr_block = "10.50.0.0/16" } # ← intended: eu-west-1
```

HINT

Resources default to the un-aliased block. Opting in takes one meta-argument. (Book 1, Chapter 17.)

ex-29 THE TASK (T3 • Read the error)

CI fails with `string required, but have number` on a `map(string)` variable fed by `TF_VAR_tags`. **Diagnose the quoting bug and write the fixed export** — then state the general rule for complex types over env vars.

```
$ export TF_VAR_tags={Env:prod}
$ terraform plan
Error: string required, but have number # ...on var.tags
```

HINT

Env vars arrive as strings; complex types need HCL syntax with quoting. (Book 1, Chapter 14 — the #1 CI variable bug.)

→ [Solutions sol-28 • sol-29](#)

Follow-up rep — count the fleet

Ex-28's DR region needs 2 instances, primary needs 3 — same config, per-provider counts. **Write the instance block** using one count expression driven by which provider serves it, without duplicating the resource.

```
resource "aws_instance" "web" {
  count = ??? # 3 in primary, 2 in DR — keyed off what?
}
```

EXAM LENS

`alias` declares, `provider = selects`. `TF_VAR_*` arrives as strings — complex types need single-quoted HCL, the #1 CI variable bug.

ex-30 drill + band checkpoint

ex-30 THE TASK (T5 • Command choice)

A `count = var.enable_monitoring ? 1 : 0` log group feeds an output. With monitoring OFF, all four candidates behave differently. **Pick the safe reader and kill the other three:**

A) `aws_cloudwatch_log_group.v[0].arn` B) `one(aws_cloudwatch_log_group.v[*].arn)` C) `aws_cloudwatch_log_group.v[*].arn[0]` D) `length(...) > 0 ? v[0].arn : null`

```
resource "aws_cloudwatch_log_group" "v" {
  count = var.enable_monitoring ? 1 : 0 # OFF → empty list
  name = "/app/verbose"
}
output "log_group" { value = ??? }
```

HINT

Empty list breaks every index — except the function built for exactly-one-or-null. D works but says in four lines what one function says. (**Book 1, Chapter 20.**)

BAND CHECKPOINT

You finish drills when your solutions match on **WHY**, not just the answer. If ex-20's index logic or ex-21's identity catch still feels fuzzy, re-read **Book 1, Chapter 20** before bosses — everything red ahead assumes keys.

→ [Solution sol-30](#)

Follow-up rep — lookup with fallback

The log-group output must read "none" (not null) when monitoring is off, and the real ARN when on. **Write it two ways:** with `one()` + fallback logic, and with `lookup()` over a one-entry map — then state which you'd keep and why.

```
output "log_group" {
  value = ??? # "none" when count = 0, ARN when count = 1
}
```

EXAM LENS

0/1 resources read via `one()` — never bare `[0]`. Tainted resources rebuild next apply; the exam asks what taint schedules.

ex-31 • ex-32 bosses

ex-31 THE TASK (T7 • Trace the race)

Two engineers share one bucket, one lock table. A applies (holds lock ID a1b2); B's apply starts 10s later; A's laptop dies mid-apply.

Narrate minute by minute: what B sees, what the table holds, the exact 4-step recovery in order — and the one thing nobody is allowed to do.

```
# t+0 A: apply (lock a1b2)   # t+10s B: apply → ???
# t+60s A laptop dead. State: half-written. Lock: held by a ghost.
# recover: read → confirm → release → prove. Never: _____
```

HINT

WHO holds it, WHEN, WHAT operation — then confirm death via human/CI/ps, release by ID, prove health. The forbidden move deletes a row by hand. (Book 1, Chapter 11.)

ex-32 THE TASK (T3 • Wrong value, live debug)

env should be prod but the plan shows staging. Layers present: default dev, terraform.tfvars (staging), z.auto.tfvars (qa), TF_VAR_env=prod, and NO -var flag. **Which layer won and why — then the two commands that prove the resolution order and the fix.**

```
$ terraform plan   # shows: env = "staging"   ← expected prod!
```

HINT

Someone's mental ladder is upside down: files beat env vars; only CLI beats files. Which file outranks which? (Book 1, Chapter 14.)

→ [Solutions sol-31 • sol-32](#)

Follow-up rep — separate the locks

Ex-31's team is tired of one shared lock serializing three envs. **Design the escape:** which backend argument changes per env, what the three resulting state keys look like, and what new operational cost this buys them (there is always a cost).

```
# backend "s3" { bucket = ???   key = ??? }
# dev → ???   staging → ???   prod → ??? (+ lock tables?)
```

EXAM LENS

Locking questions always hide a live holder. Read WHO/WHEN/WHAT, confirm death, release by ID — and remote backends + tfstate-never-in-git ride along.

ex-33 • ex-34 bosses

ex-33 THE TASK (T6 • Rename at scale)

50 count-indexed workers must become a `for_each` map keyed by hostname — same 50 machines, zero rebuilds, downstream values(`...`)[*].`id` keeps working. **Write the migration:** new block, the ledger command pattern (one example + "repeat ×50" strategy), and the downstream reference rewrite.

```
# before: aws_instance.worker[0..49] (count)
# after:  aws_instance.worker["w-01"..] (for_each)
# downstream today: aws_instance.worker[*].id
```

HINT

New keys mean new identities — plan screams rebuild. The ledger can adopt each address first; script the fifty moves, don't type them. (Book 1, Chapters 12 + 20.)

ex-34 THE TASK (T3 • The half-built box)

Apply failed INSIDE `remote-exec` (apt timeout). The instance exists; re-apply says **0/0/0**. **Diagnose the exact state of the world** (resource? software? plan's view?) and give the two recovery paths: the sledgehammer AND the architectural cure that ends this incident class.

```
$ terraform apply
aws_instance.web: Creating... Creation complete [id=i-012345]
aws_instance.web (remote-exec): ... apt timeout ... Error.
$ terraform apply # → 0 to add, 0 to change, 0 to destroy. Box: half-configured.
```

HINT

Creation-only already fired; the plan sees a finished resource. Recovery re-creates; the cure removes the hook. (Book 1, Chapter 22.)

→ [Solutions sol-33 • sol-34](#)

Follow-up rep — moved blocks

Ex-33's team dislikes fifty hand-typed state `mv` commands. **Write the declarative alternative:** a moved block for ONE worker, state exactly when Terraform honors it (plan or apply), and the one condition under which the team must still prefer CLI `mv`.

```
moved {
  from = aws_instance.worker[0]
  to   = ??? # keyed address for w-01
}
```

EXAM LENS

Identity lives in addresses: key change = new resource unless adopted. moved blocks and state `mv` are the same idea in two costumes.

ex-35 • ex-36 bosses

ex-35 THE TASK (T6 • Extract the module)

Root `main.tf` holds a 60-line VPC+subnets block used nowhere else — yet. Two more envs land next quarter. **Extract it into `modules/vpc/` without changing any real resource:** the three files, which lines move where, the root call block, and the command that proves zero drift after the move.

```
# root today: resource "aws_vpc" "main" + 3x aws_subnet (all live)
# root tomorrow: module "vpc" { source = "../modules/vpc" ... }
# state tonight: aws_vpc.main, aws_subnet.public[0..2] — must NOT rebuild
```

HINT

Moving code changes addresses (`aws_vpc.main` → `module.vpc.aws_vpc.main`) — the ledger needs adoption per resource, same trick as ex-14 but namespaced. (Book 1, Chapters 12 + 23.)

ex-36 THE TASK (T5 • Destroy with judgment)

Friday cleanup: kill the dev env (VPC + instances, no state) but the shared data VPC in the SAME state must survive — and prod state must be untouchable by construction. **Give:** the scoped destroy command, the resource-level guard that would have saved you if prod were in scope, and the workspace check typed first.

```
# one state: aws_vpc.dev + aws_instance.dev[*] + aws_vpc.data
# goal: dev gone, data alive, prod never addressable from here
```

HINT

Scope by target, guard by lifecycle, isolate by workspace — three independent safeties. (Book 1, Chapters 13 + 19.)

→ [Solutions sol-35 • sol-36](#)

Follow-up rep — prevent_destroy drill

Ex-36's Friday cleanup gains a rule: the data VPC must be UNDESROYABLE by any command short of editing code. **Write the lifecycle guard, show the exact error a scoped destroy prints when it touches it, and state the two-step override** (edit + apply) for the day decommissioning is real.

```
resource "aws_vpc" "data" {
  cidr_block = "10.10.0.0/16"
  lifecycle { ??? }
}
$ terraform destroy -target=aws_vpc.data
# Error: ???
```

EXAM LENS

`prevent_destroy` fails the plan loudly — the exam's favorite guard. `-target` scopes; workspaces isolate; all three stack.

ex-37 • ex-38 bosses

ex-37 THE TASK (T1 • The cross-VPC SG bug)

App VPC peers to data VPC. The app SG references the DB SG by ID across the peering — apply fails. **Explain why SG references die across peerings, write the working ingress block** (CIDR-based), and add the DNS flag both sides need or the app still can't resolve the DB hostname.

```
resource "aws_security_group_rule" "app_to_db" {
  security_group_id = aws_security_group.app.id
  source_security_group_id = aws_security_group.db.id # ← dies across peering
  from_port = 5432   to_port = 5432   protocol = "tcp"
}
```

HINT

SG identity doesn't cross the peering; CIDR does. And resolution needs an explicit opt-in per side. (**Book 1, Chapters 6 + 20...** and the silent-DNS section.)

ex-38 THE TASK (T4 • Wire two stacks)

Network stack exports `vpc_id` + `private_subnet_ids`; app stack needs both, separate states, network applies first. **Write both sides:** the two output blocks, the consumer data block, one resource consuming each value — and the CI ordering rule that makes it safe.

```
# network/outputs.tf: output "vpc_id" { ??? } output "private_subnet_ids" { ??? }
# app: data "terraform_remote_state" "network" { ??? }
# app resource: subnet_id = ??? (first private subnet)
```

HINT

Outputs are the contract; the data source reads another state's exports; CI graphs network → app. (**Book 1, Chapter 21.**)

→ [Solutions sol-37 • sol-38](#)

Follow-up rep — ephemeral outputs

Ex-38's app stack needs a deploy token that must NEVER land in state. **Write the output block** with the flag that keeps it out of the ledger, then state the hard consequence: when can a LATER apply still read it — and what breaks if it tries?

```
output "deploy_token" {
  value = data.aws_secretsmanager_secret_version.cur.secret_string
  ??? = true # the flag – and its price
}
```

EXAM LENS

Cross-stack = outputs publish, remote_state subscribes, CI orders. Cross-peering SGs use CIDRs plus the both-sides DNS flag.

ex-39 boss + the FINAL BOSS

ex-39 THE TASK (T7 • Trace the provider bump)

You widen `~> 5.0` → `>= 5.0`, run `plan` (not `init`), and wonder why nothing changed. **Narrate the full chain:** why `plan` ignores the edit, the command that re-resolves, what the lock file gains, and the one scenario where the widened range bites you at 3 AM.

```
# versions.tf: version = ">= 5.0"    (was "~> 5.0")
$ terraform plan  # → uses 5.31.0, the LOCKED binary. Why?
```

HINT

Constraints are wishes; the lock is law — until a specific command re-resolves. Uncapped ranges invite major-break surprises. (Book 1, Chapters 7 + 17.)

ex-40 FINAL BOSS (T4 • The full chain, cold)

No starter code. **Build the Chapter 24 chain from memory:** provider → VPC → 2 subnets → 2 instances per subnet → SG (443) → outputs (private IPs) → provisioner → destroy. Staged hints unlock in order — read each only when stuck:

```
HINT 1 (shape): for_each = toset(["a1","a2","b1","b2"]) + startswith → subnet.
HINT 2 (wiring): SG id into vpc_security_group_ids; map output over the instances.
HINT 3 (proof): ips.txt via local-exec; output -json; destroy reverses all.
```

HINT 0

Scoring: chain applies clean + output maps 4 IPs + destroy empties state = black belt. Anything less → the hint you opened names your re-read chapter.

→ Solutions sol-39 • sol-40

Follow-up rep — timed run rules

The final boss counts as passed only under exam conditions. **Copy this contract onto paper before starting:** 30 minutes, no Book 1, hints cost 5 minutes each, `plan` before every `apply`. Fail any clause and the run doesn't count — when you're done, the scorecard's last row is waiting.

```
# black-belt run: 8 resources • 4 IPs in output • empty state after destroy
# hints opened: ____ × 5 min  # total time: ____  # pass? ____
```

EXAM LENS

The chain compresses the whole Associate blueprint: `auth` → `graph` → `keys` → `outputs` → `teardown`. Narrate it cold and the scenario section is paperwork.

✓ sol-01 • sol-02 • sol-03

sol-01 Second apply: nothing to do

```
Plan: 0 to add, 0 to change, 0 to destroy.
No changes. Your infrastructure matches the configuration.
```

💡 WHY — refresh found vpc-0ab123456 with the same CIDR, so desired == known == real. An empty diff is the healthy steady state, not a broken run.

⚠️ TRAP — "0/0/0 means Terraform didn't run." It ran fully: refresh + compare, then correctly did nothing.

🎓 EXAM — idempotence: same code → same infra → empty plan.

sol-02 CLI v1.9.8 • provider v5.31.0, locked

```
Terraform v1.9.8 + hashicorp/aws v5.31.0
Lock: .terraform.lock.hcl ← written by terraform init
```

💡 WHY — terraform version prints both binaries; the lock pins exact builds + hashes so teammates download identical bits.

⚠️ TRAP — reading only "v1.9.8" and ignoring the provider line. Version bugs live in providers, not the CLI.

🎓 EXAM — lock file committed, .terraform/ ignored.

sol-03 Pillar B, three kills

```
✓ B) Platform-agnostic — one apply already spans AWS + GitHub.
X A) best practices — true, but says nothing about second APIs.
X C) community — nice, not the cost argument. X D) speed — irrelevant here.
```

💡 WHY — CloudFormation cannot manage GitHub/Cloudflare; the moment a second API enters, "free and deep" loses to "one workflow everywhere".

⚠️ TRAP — picking community: popular with managers, answers nothing about multi-cloud.

🎓 EXAM — "AWS-only?" → CloudFormation. "Multi-cloud?" → Terraform.

🔗 Variation, worked — tainted steady state

Same VPC, but someone ran `terraform taint aws_vpc.main` first. Morning apply now prints **1 to add, 0 to change, 1 to destroy** — destroy-then-create of the SAME config. Lesson: taint marks KNOWN as poisoned, so the empty diff becomes a replacement diff. Untaint (untaint) restores the 0/0/0.

```
Plan: 1 to add, 0 to change, 1 to destroy. # -/+ in disguise
```



sol-04 • sol-05 • sol-06

sol-04Parse → graph → plan → auth → VPC → subnet

```
1 HCL parsed • 2 Core builds graph (edge: subnet→VPC)
3 plan diff computed • 4 provider authenticates
5 VPC created • 6 subnet created (graph order, not file order)
```

💡 WHY — Core reads everything first, derives order from references, and only then lets plugins touch APIs. Files are unordered; the graph is the order.

⚠️ TRAP — putting auth first. Nothing authenticates before there's a plan worth executing.

🎓 EXAM — Core plans, plugins execute; order comes from references.

sol-05Leaked keys + hardcoded region

```
# fixed provider.tf – no secrets, env-driven auth
provider "aws" { region = "us-east-1" }
# $ export AWS_ACCESS_KEY_ID=... AWS_SECRET_ACCESS_KEY=...
# (or SSO profile / OIDC – never committed values)
```

💡 WHY — committed keys get scraped in minutes; env vars/SSO keep secrets out of git while the block stays minimal and portable.

⚠️ TRAP — "it's a private repo, fine." Private repos leak via clones, CI logs, and future-public flips.

🎓 EXAM — static keys ⚠️ / env vars ✓ / CLI config ✓.

sol-06resource • aws_vpc • main

```
resource "aws_vpc" "main" { # singular, underscores, lowercase
  cidr_block = "10.0.0.0/16"
} # → address aws_vpc.main.id resolves
```

💡 WHY — anatomy is fixed: keyword resource, TYPE with underscores, quoted lowercase name. The address derives from exactly these three tokens.

⚠️ TRAP — fixing two of three and re-running into the third error. Read all three tokens every time.

🎓 EXAM — resource "TYPE" "NAME"; address = TYPE.NAME.attr.



Variation, worked — the graph behind the plan

Ex-04's trace has a sequel: the subnet references `aws_vpc.main.id` AND `aws_internet_gateway.igw.id`. Draw the two edges, then state which resource the graph builds FIRST — and why file order (subnet written above VPC) still can't fool it.

```
# edges: subnet → VPC, subnet → IGW. First built: VPC + IGW (any order),
# subnet last. References are the only order that exists.
```



sol-07 • sol-08 • sol-09

sol-07aws 5.31.0 + local 2.5.1 • lock commits • binaries ignored

```
Providers: hashicorp/aws v5.31.0, hashicorp/local v2.5.1
COMMIT: .terraform.lock.hcl  IGNORE: .terraform/ (binaries)
```

💡 WHY — init output lists every installed plugin; the lock freezes exact builds; the directory is re-downloadable weight that never belongs in git.

⚠️ TRAP — committing .terraform/ (megabytes of binaries) while ignoring the tiny lock file. Exactly backwards.

🎓 EXAM — init downloads; lock pins; gitignore splits them.

sol-081 add (type swap ~) • 1 destroy (web[2])

```
~ aws_instance.web[0] # t3.micro → t3.small, same ID
~ aws_instance.web[1] # same
- aws_instance.web[2] # count shrink kills LAST index
Plan: 0 to add, 2 to change, 1 to destroy.
```

💡 WHY — type is in-place (~); count shrink always drops the highest index. Two change kinds, one tail.

⚠️ TRAP — "web[0] dies" (count never kills the first) or missing the ~ lines entirely.

🎓 EXAM — ~ in-place • - destroy • count shrinks from the top.

sol-09B, three kills

```
✓ B) "~> 5.0" — 5.x automatically, 6.0 never. The house default.
X A) ">= 5.0" — uncapped: 6.0 surprise breaks at 3 AM.
X C) "= 5.31.0" — frozen: safe but manual toil per bug-fix.
X D) none — floating: every init is a lottery.
```

💡 WHY — pessimistic pinning takes compatible fixes and refuses breaking majors: automation without surprises.

⚠️ TRAP — exact pins feel "safest" but trade breakage risk for toil; uncapped feels "modern" but trades toil for breakage.

🎓 EXAM — ~> default • = freeze • bare >= never.

Variation, worked — type swap with two changes

Ex-08's sequel: count stays 3, but the type changes AND one tag value changes on all three. The tail reads **0 to add, 3 to change, 0 to destroy** — three ~ lines, zero drama. Lesson: attribute edits are in-place across the whole set; only count edits kill indexes.

```
~ aws_instance.web[0] ~ web[1] ~ web[2]
Plan: 0 to add, 3 to change, 0 to destroy.
```



sol-10 • sol-11 • sol-12

sol-10init → plan → apply tfplan

```
1 terraform init (binaries first – nothing works without them)
2 terraform plan -out=tfplan (read-only preview + saved contract)
3 terraform apply tfplan (executes EXACTLY the reviewed plan)
```

💡 WHY — each step's output is the next step's input: binaries → contract → execution. Skipping straight to apply merges preview and execution blind.

⚠️ TRAP — apply before init (no plugins) or bare apply in prod (unreviewed changes).

🎓 EXAM — plan is read-only; apply tfplan = reviewed == executed.

sol-11Duplicate VPC — risk ①

```
# before rm: config 1 • state 1 (vpc-0ab1) • AWS 1 → plan 0/0/0
# after rm:  config 1 • state 0 • AWS 1 → plan: 1 to add (ghost!)
# after apply: config 1 • state 1 (vpc-0cd2) • AWS 2 → DUPLICATE
```

💡 WHY — with no ledger, desired-vs-known computes "everything is new." Terraform builds a second VPC beside the forgotten one.

⚠️ TRAP — "apply will re-adopt it." Nothing re-adopts except import; apply creates.

🎓 EXAM — deleting tfstate → Terraform forgets everything → duplicates.

sol-12S3 backend, two keys, one rule

```
terraform {
  backend "s3" {
    bucket = "tfstate" # prod-grade: separate buckets per env
    key    = "env/dev/terraform.tfstate" # ← per-env key (staging: env/...)
    region = "us-east-1" dynamodb_table = "tfstate-locks" encrypt = true
  }
}
# RULE: one state per env – never two envs, one file.
```

💡 WHY — separate keys give separate ledgers; separate buckets add IAM blast-radius isolation; locking serializes the team.

⚠️ TRAP — same key + different tfvars (one ledger, two writers — the original incident).

🎓 EXAM — key per env minimum; bucket per env for prod.

Variation, worked — backend keys for three envs

Ex-12's client grows a staging env. The bucket stays one; the keys become env/dev|staging|prod/terraform.tfstate. One backend "s3" block CAN'T hold three keys — so each env gets its own -backend-config file (or workspace suffix). Same ledger math, three ledgers.

```
# backend-dev.hcl → key = "env/dev/terraform.tfstate"
# backend-staging.hcl → key = "env/staging/terraform.tfstate"
```




sol-13 • sol-14 • sol-15

sol-13Held lock: verify, then force-unlock ID

```
# 1 read: holder ci-runner-4, op apply, since 10:02
# 2 confirm dead: CI run red / human says killed / ps empty
# 3 release: terraform force-unlock a1b2-c3
# 4 prove: plan clean before anyone re-applies
```

💡 WHY — the lock is evidence, not the enemy: it names who/why. Release follows proof of death, never impatience.

⚠️ TRAP — force-unlocking a LIVE holder (two writers, one ledger → corruption) or hand-deleting the DynamoDB row.

🎓 EXAM — read → confirm → release → prove. Never delete the row.

sol-14Ledger first, code second

```
$ terraform state mv aws_instance.web aws_instance.app
Move "aws_instance.web" to "aws_instance.app" - 1 moved.
# then rename in main.tf → next plan: 0/0/0 (same box, new name)
```

💡 WHY — identity lives in the address: move the ledger entry and the code rename becomes a no-op instead of destroy+create.

⚠️ TRAP — renaming code first (plan shows -/+ on a live box) or using `rm` (orphans it).

🎓 EXAM — `mv` renames tracking; `rm` abandons; neither touches AWS.

sol-15Canonical form + the gate

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
  tags       = { Name = "x" }
}
# gate: terraform fmt -check (exit 1 = unformatted, CI fails)
```

💡 WHY — two-space indent, single spaces, aligned `=:` the machine enforces style so reviews stay about logic.

⚠️ TRAP — "fmt changes behavior." It never does — pure whitespace. Fear of `fmt` is fear of nothing.

🎓 EXAM — `fmt` rewrites; `-check` gates; `-recursive` covers dirs.



Variation, worked — mv across modules

Ex-14's sequel: the instance moves into `module.cluster` instead of a plain rename. Same trick, namespaced address: `terraform state mv aws_instance.web 'module.cluster.aws_instance.web'`. The rule never changes — adopt first, edit code second, quiet plan proves it.

```
$ terraform state mv aws_instance.web 'module.cluster.aws_instance.web'
# then add the module block → plan 0/0/0.
```



sol-16 • sol-17 • sol-18

sol-16 Reference, not string + redraw

```
resource "aws_subnet" "a" {
  vpc_id = aws_vpc.main.id # ← reference creates the edge
  cidr_block = "10.0.1.0/24"
}
$ terraform graph | dot -Tsvg > graph.svg # edge reappears
```

💡 WHY — the graph is built from references, not values: a pasted ID is data, a reference is an ordering edge. Same string, different meaning.

⚠️ TRAP — adding `depends_on` to fix it. Works, but hides the real breakage: the subnet no longer follows VPC renames.

🎓 EXAM — references create edges; literals don't; graph proves it.

sol-17 Four vars, validation on env

```
# variables.tf
variable "ami" { type = string } # required: no default
variable "instance_type" { type = string default = "t3.micro" }
variable "name" { type = string default = "web-0" }
variable "env" {
  type = string
  validation { condition = contains(["dev","staging","prod"], var.env)
    error_message = "must be dev/staging/prod" }
}
```

💡 WHY — types catch shape errors at plan; required-ness forces callers to decide; validation rejects bad envs before anything builds.

⚠️ TRAP — defaulting everything (prod silently inherits dev values) or validating nothing.

🎓 EXAM — type + default + description + validation = complete declaration.

sol-18 qa wins — CLI beats all files

```
# winner: env = "qa" (the -var flag)
# proof experiment: set all four differently, plan, read the value -
# change one layer at a time to map the full ladder.
# fix: drop the -var flag (or set it to prod) to get prod.
```

💡 WHY — ladder is `-var > tfvars > env > default`: later-listed sources override. The stray flag outranks every file.

⚠️ TRAP — "TF_VAR beats files" (upside-down ladder — the classic). Files always beat env vars.

🎓 EXAM — full ladder: `default < TF_VAR_* < tfvars < auto.tfvars < -var-file < -var`.

Variation, worked — auto.tfvars ambush

Ex-18's sequel: a forgotten `z.auto.tfvars` setting `env = "qa"` sits in the repo. With NO `-var` flag, the winner is **qa** (auto.tfvars beats terraform.tfvars beats env). Lesson: when values surprise you, `grep -r auto.tfvars` before doubting the ladder.

```
# no -var flag + z.auto.tfvars(env="qa") → winner: "qa"
```



sol-19 • sol-20 • sol-21

sol-19 Sizing map + three workspaces + typed check

```
locals {
  sizing = { dev = { count=1 type="t3.micro" }
    staging = { count=2 type="t3.small" } prod = { count=3 type="t3.small" } }
  current = local.sizing[terraform.workspace]
}
resource "aws_instance" "cluster" {
  count = local.current.count instance_type = local.current.type }
# $ workspace new dev/staging/prod → select → apply each
# RULE: type `workspace show` (or show) before EVERY apply.
```

💡 WHY — the workspace name becomes a lookup key: same code, per-env shape, per-env state. The typed check defeats "wrong env" applies.

⚠️ TRAP — `local.sizing[var.env]` with no workspace (one state, three liars) or forgetting the default-workspace entry.

🎒 EXAM — `terraform.workspace` keys maps; state isolation comes free.

sol-20 `web[2]` dies, attachment errors

```
- aws_instance.web[2] # LAST index dies
Plan: 0 to add, 0 to change, 1 to destroy.
# downstream: Error: Invalid index - web[2] no longer exists.
# for_each wouldn't: keys hold, only the removed key dies.
```

💡 WHY — count identity IS position: shrink drops the tail, and every `[2]` reference dangles. Keys decouple identity from position.

⚠️ TRAP — "it destroys `web[0]`" or "references update themselves." Neither: tail dies, refs break.

🎒 EXAM — count shrinks from the top; dangling indexes error, never re-point.

sol-21 AZ-keyed map + mv each address

```
resource "aws_subnet" "public" {
  for_each = { for i, az in var.azs : az => i }
  cidr_block = cidrsubnet("10.0.0.0/16", 8, each.value)
  availability_zone = each.key
}
$ terraform state mv 'aws_subnet.public[0]' 'aws_subnet.public["us-east-1a"]'
# x3 - then downstream: values(aws_subnet.public)[*].id
```

💡 WHY — new keys mean new identities, so plan proposes rebuilds; adopting each address into its key first turns the refactor into a no-op.

⚠️ TRAP — applying the rebuild ("it's just subnets" — new subnets means new IPs, new routes, moved workloads).

🎒 EXAM — key change = identity change; mv preserves, apply rebuilds.

Variation, worked — splat after migration

Ex-21's downstream ALB attachment also needs rewriting: `web[2].id` is dead. For a `for_each` map the single becomes `aws_instance.web["web-c"].id` and the whole fleet becomes `values(aws_instance.web)[*].id`. Brackets hold keys now, never positions.

```
# single: aws_instance.web["web-c"].id
# all:    values(aws_instance.web)[*].id
```



sol-22 • sol-23 • sol-24

sol-22 Flag it + pipe it, never print it

```
output "db_password" { value = aws_db_instance.main.password
  sensitive = true } # hides CLI display
$ terraform output -raw db_password | aws secretsmanager create-secret ...
```

💡 WHY — sensitive masks display; piping moves the value machine-to-machine with no eyeballs. The two violations were unflagged output + human-readable transport.

⚠️ TRAP — "sensitive encrypts state." Masking only — state stays plaintext; Vault/SSM is the encryption story.

🎓 EXAM — sensitive = display; secret store = transport; Vault = rest.

sol-23 Creation-only fired long ago

```
# why nothing ran: provisioner fired at creation; edits don't re-fire it.
# recovery: terraform taint aws_instance.web → apply (rebuild + rerun)
# cure: user_data / Packer AMI / Ansible — remove the hook entirely.
```

💡 WHY — Terraform tracks the resource, not the script's effects: unchanged resource means no lifecycle event, so no hook. "Broken" is the design working.

⚠️ TRAP — re-applying harder, or adding a second provisioner on top. Both preserve the half-configured box.

🎓 EXAM — provisioners: creation-only, last resort, state-blind.

sol-24 main/variables/outputs + two inputs

```
# modules/vpc/{ main.tf, variables.tf, outputs.tf }
variable "cidr" { type = string } # per-env CIDR in
variable "azs" { type = list(string) } # per-env AZs in
module "prod_vpc" {
  source = "../../modules/vpc" # registry: source + version pin
  cidr   = "10.20.0.0/16" azs = ["us-east-1a", "us-east-1b"]
}
```

💡 WHY — the interface (two inputs, documented outputs) kills the copy-paste drift class: every env consumes the same tested shape.

⚠️ TRAP — unpinned git source (floating) or stuffing env logic INSIDE the module (unkillable coupling).

🎓 EXAM — modules package; inputs vary; pins freeze.

Variation, worked — ephemeral token output

Ex-22's sequel: a deploy token must NEVER persist. `ephemeral = true` keeps it out of state entirely — readable in the same apply, gone afterwards. Price: no later apply can re-read it. Secrets you re-read → sensitive; one-shot proofs → ephemeral.

```
output "deploy_token" { value = data.aws_secretsmanager_secret_version.cur.secret_string
  ephemeral = true } # same-apply read only
```



sol-25 • sol-26 • sol-27

sol-25~ on the SG, apply or revert

```
~ aws_security_group.web # ingress: +22-open-by-hand
Plan: 0 to add, 1 to change, 0 to destroy.
# keep it: add 22 to code → apply. revert it: console-delete 22 → plan 0/0/0.
```

💡 WHY — refresh imports REAL into the diff: hand edits surface as ~ changes against code. The plan is the drift report you didn't ask anyone to write.

⚠️ TRAP — "plan will ignore console edits." Refresh exists precisely so it can't.

🎓 EXAM — drift shows as ~; resolve by code (apply) or console (revert).

sol-26B previews, C writes, A retires

```
✓ job 1: plan -refresh-only -out=drift.plan (preview, zero writes)
✓ job 2: apply -refresh-only (deliberate state sync)
X A) refresh - legacy standalone; same engine, worse manners.
```

💡 WHY — the flag form says what it does: preview vs write, each explicit and reviewable. Bare refresh writes silently with no plan artifact.

⚠️ TRAP — refreshing to "fix" drift (it blesses the drift into state instead of removing it).

🎓 EXAM — -refresh-only previews or syncs; bare refresh is legacy.

sol-27Write → import → quiet plan

```
# 1 resource "aws_vpc" "main" { cidr_block = "10.0.0.0/16" }
$ terraform import aws_vpc.main vpc-0ab123456
Import successful!
$ terraform plan # → 0 to add, 0 to change, 0 to destroy ✓
```

💡 WHY — import binds identity only: config declares intent, import points at reality, the quiet plan proves they describe the same object.

⚠️ TRAP — importing before writing config (binds, then plan proposes to fill the missing attributes destructively).

🎓 EXAM — write → import ADDR ID → plan 0/0/0.

Variation, worked — JSON plan interrogation

Ex-25's drift deserves a machine-readable proof for CI: `terraform show -json drift.plan | jq '.resource_changes[] | select(.change.actions != ["no-op"]) | .address'` lists every drifted address. Empty output = clean bill; any line = page the human who owns that resource.

```
$ terraform show -json drift.plan | jq -r '.resource_changes[].address'
aws_security_group.web # ← the drifter, by address
```



sol-28 • sol-29 • sol-30

sol-28 provider = aws.eu on the DR block

```
resource "aws_vpc" "dr" {
  provider    = aws.eu    # ← the missing line: opt into the alias
  cidr_block = "10.50.0.0/16" # now lands eu-west-1, no overlap
}
```

💡 WHY — resources default to the un-aliased block; without the opt-in both VPCs share a region and the "clearly regional" code lies.

⚠️ TRAP — "the alias on the provider is enough." Declaring alias creates the option; only `provider =` exercises it.

🎓 EXAM — alias declares, `provider =` selects, default is un-aliased.

sol-29 Quote it as HCL, not shell

```
$ export TF_VAR_tags='{ "Env": "prod" }' # single-quoted HCL map
# rule: primitives auto-convert; lists/maps/objects need HCL syntax.
```

💡 WHY — env vars arrive as raw strings: 5 converts cleanly but `{Env:prod}` parses as garbage the type system rejects.

⚠️ TRAP — double quotes (shell eats them) or unquoted braces (shell expands them). Single-quote the HCL.

🎓 EXAM — `TF_VAR_*` = strings; complex types = single-quoted HCL.

sol-30B, three kills

```
✓ B) one(v[*].arn) - empty → null, single → value. Built for this.
✗ A) [0] - Invalid index on empty. ✗ C) [*][0] - same crash, extra steps.
~ D) length guard - correct but four lines for one function's job.
```

💡 WHY — `one()` encodes exactly-one-or-null: the cardinality contract of every 0/1 resource in one call.

⚠️ TRAP — C feels "splat-safe." Splat of empty is empty; indexing empty is still an error.

🎓 EXAM — 0/1 resources → `one()`; never bare `[0]`.

📦 Variation, worked — version pin in the fix

Ex-28's sequel: the team pins the provider properly this time — `version = "~> 5.0"` in `required_providers` plus the alias block. Fresh init installs 5.x in both regions; the 6.0 morning from the follow-up rep can never recur. Pins are incident prevention.

```
# required_providers: version = "~> 5.0" → 6.0 cannot install. Ever.
```



sol-31 • sol-32 • sol-33

sol-31B waits, ghost released by the book

```
# t+10s B: Error acquiring lock (held: a1b2, ci-runner... wait — A, human)
# table: one row, LockID=state, holder A. B retries or stands down.
# recovery: ① show/CI/ps confirm A dead ② force-unlock a1b2
# ③ plan clean ④ B applies. NEVER: hand-delete the DynamoDB row.
```

💡 WHY — the lock serializes writers: B's error IS the system working. Recovery re-establishes single-writer truth before anything proceeds.

⚠️ TRAP — unlocking while A might be alive (split-brain ledger) or deleting the row (destroys the evidence AND the guard).

🎓 EXAM — one lock, one writer; ghosts released by ID after proof.

sol-32terraform.tfvars won (staging); env loses to files

```
# winner: staging — terraform.tfvars outranks TF_VAR_env.
# prove: TF_VAR_env=x plan → staging. rm terraform.tfvars → prod (env).
# rm z.auto.tfvars too → still prod (env beats default dev).
# fix: set -var="env=prod" (CLI beats all) or correct the tfvars.
```

💡 WHY — the ladder never inverts: files always outrank env vars, CLI outranks files. "Expected prod" assumed env-first; the machine is file-first.

⚠️ TRAP — deleting tfvars AND auto.tfvars and expecting dev (env still stands between files and default).

🎓 EXAM — debug values by removing layers top-down: -var, files, env, default.

sol-33Adopt fifty addresses, rewrite one reference

```
$ for i in $(seq -w 1 50); do
  terraform state mv "aws_instance.worker[$((10#$i-1))]" "aws_instance.worker[\w-$i\]"
done
# downstream: values(aws_instance.worker)[*].id
# plan after: 0/0/0 — same boxes, keyed identities.
```

💡 WHY — identity moves with adoption: each mv re-points one address before the code switch, so the refactor never proposes a rebuild.

⚠️ TRAP — hand-typing fifty moves (typo = orphan + duplicate) or applying the rebuild ("faster" — fifty new boxes, fifty dead ones).

🎓 EXAM — bulk renames are scripted mv's, verified by a quiet plan.

Variation, worked — lock table per env

Ex-31's sequel at scale: three envs, three DynamoDB tables (tfstate-locks-dev|staging|prod). Dev's stuck apply can no longer block prod's deploy — at the price of three tables to guard and three lock rows to read during incidents. Isolation always invoices.

```
# dynamodb_table = "tfstate-locks-${terraform.workspace}" — per-env guard
```



sol-34 • sol-35 • sol-36

sol-34 Taint the box, bake the cure

```
# world: resource EXISTS (i-012345, half-configured) • plan sees done.
# sledgehammer: terraform taint aws_instance.web → apply (rebuild + rerun)
# cure: Packer AMI / user_data / Ansible – delete the provisioner.
```

💡 WHY — creation-only fired once and will never re-fire: the plan is green because the RESOURCE is complete; the software was never Terraform's to track.

⚠️ TRAP — re-applying expectantly, or on_failure = continue as a fix (hides the next failure too).

👉 EXAM — green plan + broken box = fired hook; taint recovers, baking cures.

sol-35 Move code, adopt namespaced addresses

```
# modules/vpc/{ main.tf ← block, variables.tf ← cidr/azs, outputs.tf }
module "vpc" { source = "../modules/vpc" cidr = "10.0.0.0/16" ... }
$ terraform state mv aws_vpc.main 'module.vpc.aws_vpc.main'
$ terraform state mv 'aws_subnet.public[0]' 'module.vpc.aws_subnet.public["us-east-1a"]'
# ...per resource, then plan 0/0/0.
```

💡 WHY — module addresses namespace everything: adoption re-points each ledger entry ahead of the code move, so extraction is bookkeeping, not rebuild.

⚠️ TRAP — moving code first (plan proposes destroy+create of the network — downtime by refactor).

👉 EXAM — extraction = move code + mv each address + quiet plan.

sol-36 Show, target, guard — prod unreachable

```
$ terraform workspace show # → dev. If prod: STOP.
$ terraform destroy -target=aws_vpc.dev -target='aws_instance.dev[*]'
# guard (already in code, hopefully):
resource "aws_vpc" "data" { ... lifecycle { prevent_destroy = true } }
```

💡 WHY — three independent safeties: eyes (show), scope (target), code (prevent_destroy). Prod lives in another workspace/state, unreachable from here by construction.

⚠️ TRAP — naked destroy -auto-approve in a shared state (takes data with dev) or trusting memory over show.

👉 EXAM — show → target → guard; prod isolation is structural, not carefulness.

🔗 Variation, worked — user_data instead

Ex-34's box gets rebuilt the right way: the apt lines move into user_data, which re-runs from a clean image on every replacement and shows in the plan as config — tracked, reviewable, retryable. Compare: provisioner (blind, once) vs user_data (visible, every boot).

```
user_data = <<-EOT
#!/bin/bash
apt-get install -y nginx # ← runs EVERY boot, visible in config
EOT
```


✓ sol-37 • sol-38 • sol-39

sol-37CIDR, not SG refs + both-side DNS

```
ingress {
  from_port = 5432  to_port = 5432  protocol = "tcp"
  cidr_blocks = ["10.20.0.0/16"] # ← data VPC CIDR, not the SG id
}
# + aws_vpc_peering_connection_options BOTH sides:
# allow_remote_vpc_dns_resolution = true
```

💡 WHY — SG identity stops at the peering boundary, but routes carry CIDRs fine; DNS needs its own explicit opt-in per side or names stay dark.

⚠️ TRAP — opening 0.0.0.0/0 "to make it work" (works, and invites the internet to the database).

🎓 EXAM — cross-peering: CIDR sources + both-side DNS flag.

sol-38Contract out, data source in, graph ordered

```
# network/outputs.tf
output "vpc_id" { value = aws_vpc.main.id }
output "private_subnet_ids" { value = aws_subnet.private[*].id }
# app/data.tf
data "terraform_remote_state" "network" {
  backend = "s3"
  config = { bucket = "tfstate-prod" key = "network/terraform.tfstate" }
}
resource "aws_instance" "app" {
  subnet_id = data.terraform_remote_state.network.outputs.private_subnet_ids[0]
}
# CI: network job → app job (needs: network).
```

💡 WHY — outputs publish the contract, the data source subscribes, and the pipeline orders what Terraform cannot: separate states have no shared graph.

⚠️ TRAP — applying app first (null reads) or hardcoding the VPC ID (drift-proof coupling).

🎓 EXAM — outputs publish · remote_state subscribes · CI orders.

sol-39Lock rules until init -upgrade

```
# why plan ignored it: 5.31.0 satisfies ">= 5.0" — lock still valid.
$ terraform init -upgrade # re-resolves → lock gains 5.x newest + hashes
# 3 AM scenario: provider 6.0 ships breaking rename → next -upgrade
# (or fresh clone) pulls it silently. Cap stays off only with "~>".
```

💡 WHY — constraints are consulted at resolve time (init), not plan time: a satisfied lock means zero re-resolution, whatever the file says.

⚠️ TRAP — editing versions.tf and expecting plan to notice. Files wish; locks command.

🎓 EXAM — constraints resolve at init; -upgrade re-resolves; caps prevent majors.

🔗 Variation, worked — outputs for the app tier

Ex-38's network stack grows a third export — `rds_endpoint` — and the app's DB config reads it the same way:
`data.terraform_remote_state.network.outputs.rds_endpoint`. Rule of the contract: every new cross-stack need is a new output + a new consumer line, never a pasted string.

```
output "rds_endpoint" { value = aws_db_instance.main.endpoint }
```



sol-40 — the full chain, worked

```

provider "aws" { region = "us-east-1" } # (+ emulator creds/endpoints)
resource "aws_vpc" "main" { cidr_block = "10.0.0.0/16" }
resource "aws_subnet" "a" { vpc_id = aws_vpc.main.id
  cidr_block = "10.0.1.0/24"    availability_zone = "us-east-1a" }
resource "aws_subnet" "b" { vpc_id = aws_vpc.main.id
  cidr_block = "10.0.2.0/24"    availability_zone = "us-east-1b" }
resource "aws_security_group" "web" { # 443 in, all out
  vpc_id = aws_vpc.main.id
  ingress { from_port=443 to_port=443 protocol="tcp" cidr_blocks=["0.0.0.0/0"] }
  egress { from_port=0 to_port=0 protocol="-1" cidr_blocks=["0.0.0.0/0"] } }
resource "aws_instance" "web" {
  for_each = toset(["a1","a2","b1","b2"])
  subnet_id = startswith(each.key,"a") ? aws_subnet.a.id : aws_subnet.b.id
  vpc_security_group_ids = [aws_security_group.web.id]
  provisioner "local-exec" { command = "echo ${each.key}=${self.private_ip} >> ips.txt" } }
output "ips" { value = { for k,i in aws_instance.web : k => i.private_ip } }
# $ init → plan (8 add) → apply → output shows 4 IPs → destroy → state empty

```

💡 WHY — every chapter fires once: provider auth, graph-ordered VPC→subnets, SG by reference, keyed fleet, map output, creation-only hook, reverse-order teardown. The chain is the course.

⚠️ TRAP — count instead of for_each (works until anything renames), SG by CIDR inside one VPC (works, and teaches nothing), skipping destroy ("done" with live orphans).

🎓 EXAM — narrate this chain cold: 8 resources, 4 IPs, empty state at the end.



Variation, worked — destroy in reverse

Ex-40's finale deserves its mirror: destroy runs the chain backwards — instances, SG, subnets, VPC — each Destroying...

Destruction complete in reverse dependency order. The empty state afterwards is the second proof (the first was the 4 mapped IPs). Both proofs, or it didn't happen.

```

aws_instance.web["b2"]: Destruction complete → ... → aws_vpc.main: Destruction complete
Destroy complete! Resources: 8 destroyed.

```

Scorecard — band × topic grid

Tick honestly: **green** solved cold · **amber** needed the hint · **red** read the solution. Any red returns to next week's rotation with its Book 1 chapter.

Ex	Topic (Book 1 ch)	Type			
01–05	laC · facts · pillars · core · creds	T2 T3 T5 T7 T1	☐	☐	☐
06–10	VPC · init · plan · pins · CLI order	T1 T3 T2 T5 T5	☐	☐	☐
11–15	state · backend · lock · mv · fmt	T7 T4 T3 T6 T1	☐	☐	☐
16–20	graph · vars · precedence · envs · count	T3 T6 T2 T4 T2	☐	☐	☐
21–25	keys · secrets · modules · drift · refresh	T6 T3 T4 T7 T5	☐	☐	☐
26–30	import · alias · env types · one()	T5 T4 T1 T3 T5	☐	☐	☐
31–35	race · debug · scale · failure · module	T7 T3 T6 T3 T6	☐	☐	☐
36–40	destroy · peering · stacks · bump · CHAIN	T5 T1 T4 T7 T4	☐	☐	☐

♥ GRADUATION BAR

Warm-ups 13/15 cold · drills match on WHY · bosses all green · final chain from memory with 4 IPs and an emptied state. Then the Associate exam is paperwork.

Back to [Training map](#) · [Re-train warm-ups](#) · [Re-fight the boss](#)